Reviewer Pack — Minimal Rank of Hodge Diamond Invariants for Monotone Circui…

Entry 987fd19025d0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 10:26:28 UTC

Minimal Rank of Hodge Diamond Invariants for Monotone Circuit Satisfiability

Entry ID: `987fd19025d0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 10:26:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Boolean Circuit Satisfiability

Statement:

> ['For any monotone circuit C computing k-CLIQUE, the Hodge diamond invariant HD(C) of its associated complex C is upper-bounded by O(n^{k}{1/6})) and lower-bounded by Ω(2^k).', 'If C is a monotone circuit with n inputs, then HD(C) ≤ O(n^{k}{1/6})).', 'For any k-CLIQUE instance I, if I can be solved by a monotone circuit C of size O(2^k * n^k), then HD(C) ≥ Ω(2^k).']

Rationale (proposer's reasoning):

> ['Hodge theory provides a rich framework for studying algebraic geometry, which could offer new insights into the structure of boolean functions.', 'The Hodge diamond invariant is a combinatorial object that encodes information about the cohomology groups of an associated complex. This invariant has been studied in various contexts but not yet applied to monotone circuit complexity.', 'If this conjecture holds, it would provide a new approach for proving lower bounds on monotone circuits and could potentially lead to breakthroughs in complexity theory.']

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c43687b9a91e99db`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The Hodge diamond invariant HD(C) for a monotone circuit C computing k-CLIQUE is within the bounds O(n^{k}{1/6})) and Ω(2^k).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge diamond invariants" AND "monotone circuit satisfiability" - "Algebraic Geometry" AND " Boolean Circuit Satisfiability" AND "circuit complexity" - "minimal rank of Hodge diamond invariants" AND monotone circuits AND k-CLIQUE

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique_instance(n, k):
        if n < k:
            return None
        vertices = list(range(n))
        edges = []
        for i in range(k):
            for j in range(i+1, k):
                edges.append((vertices[i], vertices[j]))
        remaining_vertices = [v for v in vertices if v not in vertices[:k]]
        for _ in range(math.comb(n-k, 2)):
            u, v = random.sample(remaining_vertices, 2)
            edges.append((u, v))
        return (vertices, edges)

    def monotone_circuit(C):
        # Placeholder function to simulate the construction of a monotone circuit
        # This is a dummy implementation and should be replaced with actual logic
        return set()

    def hodge_diamond_invariant(C):
        # Placeholder function to compute the Hodge diamond invariant
        # This is a dummy implementation and should be replaced with actual logic
        return 0
```

```
    n = random.randint(5, 40)
    k = random.randint(2, min(n-1, 5))
    instance = generate_k_clique_instance(n, k)
    if instance is None:
        return {
            "metric_name": "Hodge Diamond Invariant",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "k too large for n"
        }

    vertices, edges = instance
    C = monotone_circuit(C)
    HD_C = hodge_diamond_invariant(C)

    return {
        "metric_name": "Hodge Diamond Invariant",
        "metric_value": HD_C,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_d = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len(results)
    std_d = math.sqrt(sum((r["metric_value"] - mean_d) ** 2 for r in results if r["instances_teste
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["instances_tested"] == 0 for r in results):
        print("RESULT: INCONCLUSIVE no instances tested")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_d} std={std_d} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_faili
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67024de7.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67024de7.py", line 58, in run_trial
    C = monotone_circuit(C)
                         ^
UnboundLocalError: cannot access local variable 'C' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that neither the support nor falsification conditions could be evaluated. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12958
2	preregistration	ollama_remote	glm4:latest	0	0	5626
3	novelty	ollama_remote	glm4:latest	0	0	4783
4	novelty	ollama_remote	glm4:latest	0	0	5481
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17889
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11914
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12410
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8733
9	judge	ollama_remote	glm4:latest	0	0	20532

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 100325 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/987fd19025d0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/987fd19025d0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/987fd19025d0.tar.gz` (if generated)